How Far Can We Trust Python Call Graphs? Static Analysers, LLMs and Runtime Ground Truth on Real Repositories

Sankalp Shankar
Dept. of AI & ML, Acharya Institute of Technology, Bangalore, India
sankalps.23.beai@acharya.ac.in

***Abstract*—Call graphs underpin code navigation, refactoring tools and, increasingly, the retrieval layer of LLM coding assistants. For Python they come from static analysers whose accuracy is mostly reported on micro-benchmarks. We measure two static analysers, PyCG and a Pyright-based extractor, on real repositories sampled from GitHub, and score them, together with three open-weight LLMs, against the calls each project's own test suite actually makes. On 105 repositories, PyCG exceeded a 30-minute budget on 21% of them and, on 46% of those it finished, produced a different call graph for a different hash seed, with edge counts varying by up to 65%; Pyright completed every repository it was run on. The two analysers share a median of 0.65 of their edges. Against runtime ground truth on 20 repositories, the union of both analysers contains a median of 0.78 of the calls observed (range 0.45–0.94), and 80% of the calls it misses go through a method name shared by several functions. LLM resolvers raise recall on hard items from 0.67 to 0.77–0.85 but lower precision from 0.88 to 0.66–0.75; they find calls static analysis misses and claim calls that never happen, and a learned combination of both does not generalise to unseen projects. We release the pinned corpus, the pipelines and all measurements.**

***Index Terms*—call graphs, static analysis, dynamic analysis, Python, large language models, empirical study**

I. Introduction

A call graph records which functions call which. Tools for program comprehension, refactoring, dead-code detection and vulnerability reachability are built on one, and code-graph retrieval for LLM agents now uses it to decide which code a model sees [1]. Python makes call graphs hard to build statically: dispatch depends on runtime types, functions are passed around as values, and decorators, properties and registries hide calls behind syntax that does not look like a call.

PyCG [2] is the standard Python call-graph generator and the baseline in recent benchmarks [3] and in studies of LLMs as static analysers [4]. Its accuracy, like that of its successors [5], is mostly reported on hand-written micro-benchmarks, where every call has a known answer. Real repositories are larger, messier and have no answer key. For Java, executing programs to bound the recall of static call graphs is established practice [6], [7]; for Python, DyPyBench compared PyCG with dynamic call graphs on 50 projects and found 51% of the dynamic edges missing [3], but attributed 59% of the missing callees to methods of built-in types and about 12% to naming differences, rather than to analysis gaps. We exclude both by construction (Section III-B).

This paper asks how far Python call graphs can be trusted on real code, in four questions:

- **RQ1 (reliability):** Does a widely used analyser produce a call graph at all, and the same one twice?
- **RQ2 (agreement):** How much do two independent static analysers agree on the same code?
- **RQ3 (recall):** Of the calls a project's tests actually make, how many do static analysers find, and where do they miss?
- **RQ4 (LLMs):** Can LLMs resolve the calls static analysis misses, and at what cost in false claims?

Our contributions are:

1) A reliability study of PyCG on 105 real repositories: it fails on 21% within 30 minutes, and its output depends on `PYTHONHASHSEED` on 46% of the rest.
2) An agreement study of PyCG and a Pyright-based extractor, mapped onto one shared node set.
3) A runtime-grounded evaluation of static and LLM call resolution on real repositories, in which every label is an observed call, not a judgement.
4) A pinned, stratified corpus pipeline (discovery, dynamism scoring, automated build triage with recorded image digests and dependency locks) and all data, released for re-measurement.

II. Related Work

Python call graphs. PyCG [2] computes an assignment graph over modules with a fixpoint and reports high precision and moderate recall on its micro-benchmark. Jarvis [5] adds flow sensitivity and reports higher precision, recall and speed than PyCG. Both evaluate primarily on micro-benchmarks and a handful of applications. We use PyCG because it remains the most common baseline, and we report its failure modes at scale.

Static versus dynamic call graphs. Sui et al. [6] and Helm et al. [7] execute Java programs to bound the recall of static call graphs and attribute misses to language features. DyPyBench [3] provides 50 executable Python projects and compares PyCG to DynaPyt-based dynamic call graphs. We differ in measuring a second static analyser on the same node

set, in stratifying the corpus by dynamism before sampling, and in evaluating LLM resolvers against the same runtime labels.

LLMs as static analysers. Venkatesh et al. [4] evaluate 24 LLMs on call-graph and type-inference micro-benchmarks and find PyCG ahead of every LLM on call graphs. We revisit the question on real code with execution-derived labels.

III. Study Design

A. Corpus

We searched GitHub for Python repositories with at least 50 stars, a push after August 2023, a `pyproject.toml` or `setup.py`, and a test suite. From the 98,416 matches we drew a seeded random sample (not ranked by stars), froze each repository's commit on the day of discovery, and scored its source with an AST-based dynamism score: weighted counts of dynamic constructs (`getattr` with a non-literal name, dynamic imports, `eval`/`exec`, registries, decorators) per thousand lines, with thresholds calibrated on a 14-repository reference set. Repositories were binned LOW, MED or HIGH and interleaved into a 558-repository queue, so that stopping early still leaves balanced strata.

An automated triage worked the first 240 repositories of the queue. Each was installed into a pinned `python:3.12` image (recorded by digest), falling back to 3.11 and 3.10 on interpreter or dependency errors, and its tests were collected with `pytest`. Every attempt records the resolved commit, the image digest and a hash of the frozen dependency list. 118 repositories built and collected at least one test; 64 built but collected none. After a licence gate, 105 entered static extraction.

B. Nodes and edges

Both analysers are mapped onto one node set, inventoried from the AST: modules, classes' bodies, functions, methods and lambdas, identified by file path and qualified name (line numbers are not part of identity). Comprehensions fold into their enclosing function; a method call resolves to the defining class in the method resolution order. An edge is *in-universe* when both caller and callee are callables of the repository itself; calls into builtins, the standard library and third-party packages are excluded, because no call-graph tool claims to model them and including them would inflate every miss rate.

C. Static analysers

PyCG 0.0.8 runs three times per repository, with `PYTHONHASHSEED` set to 0, 1 and 42. Each run uses its own container, an 8 GB memory cap and a 1800 s timeout, with the repository mounted read-only at the same fixed path, so the hash seed is the only input that changes between runs. The PyCG graph of a repository is the union of the seeds that finished.

Pyright 1.1.411 [8] has no batch call-graph mode. For every call site in the AST we ask `pyright-langserver` for the definition of the callee identifier and map the answer back to a node. Pyright runs without the repository's dependencies installed, which can only weaken it.

D. Runtime ground truth

For RQ3 and RQ4 we drew 20 of the 78 repositories where both analysers finished (8 HIGH, 7 MED, 5 LOW): admitted on Python 3.12, at least 50 tests collected, a single top-level package, no heavy native stack, a test suite that needs no external service, and preferably a deterministic PyCG (7 of the 20 are not; their PyCG graph is the union of the three seeds). Each repository's own test suite runs under a tracer built on Python 3.12's `sys.monitoring` [9]. It records every call between callables of the package, with the line it came from; calls from test code are attributed to a synthetic entry node, and a call that passes through library code (a callback handed to the standard library) is charged to the nearest package frame. The labels are therefore a lower bound on the program's behaviour: they cover what the tests execute and nothing else.

E. Call-resolution items

An *item* is a pair (caller, candidate) in which the candidate's name appears on a line of the caller that executed under the tests: as a call, as a bare reference (a property read, a function passed on) or as a class name (reaching its constructor). The label is whether the call was observed. A resolver answers yes or no for each item: *name match* (every candidate is a call, as a text search would assume), PyCG, Pyright, their union, and the LLMs. Calls whose target never appears by name in the caller cannot be items; they are counted separately so they do not disappear from anyone's recall.

F. LLM resolvers and a combiner

We query gpt-oss-20b, gpt-oss-120b and qwen3.8-27b, open-weight models served by Groq, at temperature 0. Each prompt shows one caller's executed lines, the definitions of all its candidates and the class hierarchy among them, and a JSON schema restricts the answer to candidate identifiers. Because of API rate limits, the LLMs answer a seeded sample of callers that contain at least one hard item (a name shared by several functions, a bare reference, or a pair the static analysers get wrong); every resolver is scored on the same sample. Each corpus repository contributes at most 25 sampled callers; `click` and `toolz`, traced earlier, contribute 140 and 60.

A logistic-regression combiner uses ten features: each static analyser's and each LLM's answer, how the name is written (call, reference, class name), whether it is shared by several functions, and whether the candidate is a method of the caller's own class. It is scored out-of-fold, once with folds grouped by caller function, so that no function's items are scored by a model trained on them, and once leaving out one whole repository at a time.

IV. Results

A. RQ1: Reliability of PyCG

Table I summarises static extraction. Pyright finished every repository whose package layout was detected. PyCG failed on 22 of 105 (21%): on each, all three seeds hit the 1800 s timeout. When PyCG finishes it is fast (median 0.7 s per run,

TABLE I
STATIC EXTRACTION OUTCOMES

Repositories attempted	105
both analysers finished	78
PyCG timed out on all three seeds	22
package layout not detected	5
PyCG output depends on hash seed (of 78)	36 (46%)
LOW / MED / HIGH dynamism	9/22 / 13/23 / 14/33
median / maximum edge-count spread	1.0% / 65%
repositories with spread $\geq$ 10%	7

maximum 582 s), so the failures are not slow runs cut short; PyCG either converges quickly or not within half an hour. Size is part of the explanation, since failed repositories are larger (median 935 callable nodes against 181), but not all of it: `chevron`, with 16 callable nodes, timed out on every seed. The failure rate matches DyPyBench, where PyCG produced a call graph for 39 of 50 projects [3].

On 36 of the 78 repositories where PyCG finished (46%), the three seeds produced different in-universe edge sets. Because the repository path, image and tool version were identical across runs, the hash seed alone is responsible: PyCG's fixpoint iterates over hash-ordered sets, and set iteration order in Python depends on `PYTHONHASHSEED`, which is random per process unless pinned. The effect is usually small (median spread 1.0% of edges) but has a long tail: 7 repositories vary by 10% or more, and on `movingpandas` the edge count varies by 65% between seeds. The problem occurs in every stratum, so it is not confined to highly dynamic code. It had been noticed informally by tool builders [10]; we are not aware of a measurement of how often it occurs.

Implication. A PyCG call graph is a sample from a distribution. Studies that use PyCG should pin and report `PYTHONHASHSEED`, or run several seeds and report the spread, and should budget for PyCG not terminating on roughly one repository in five.

B. RQ2: Agreement between static analysers

On the 78 repositories where both finished, the Jaccard similarity of the two analysers' in-universe edge sets has a median of 0.65 (interquartile range 0.56–0.80); 16 repositories fall below 0.5 and 19 reach 0.8 or more. Of the 33,368 edges found by either tool, 11% are found only by PyCG and 27% only by Pyright. Two analysers reading identical source disagree on roughly a third of the call graph. Agreement says nothing about which tool is right; that requires the runtime labels of RQ3.

C. RQ3: Static recall against runtime calls

We traced the test suites of 20 of the sampled repositories, stratified by dynamism, plus `click` and `toolz` from the scorer's reference set. Two could not be measured: `kolla-ansible`'s tests never import its own package, and `wsgidav`'s suite produced no countable test results under the harness. Across the remaining 20 repositories the tests exercised 5,436 distinct calls between functions and methods of the packages. Table II lists each repository.

TABLE II
STATIC RECALL AGAINST CALLS OBSERVED UNDER EACH TEST SUITE. PAIRS: DISTINCT CALLER→CALLEE CALLS OBSERVED. EDGE COLUMNS: SHARE OF THEM IN EACH STATIC GRAPH. ITEM: RECALL OF THE UNION ON CALL-RESOLUTION ITEMS.

Repository	S	Pairs	PyCG	Pyright	Union	Item
`ShivyC`	L	1459	0.34	0.41	0.45	0.46
`rlm`	L	142	0.34	0.79	0.79	0.85
`pr-agent`	L	619	0.66	0.79	0.79	0.95
`sqllineage`	M	761	0.29	0.44	0.45	0.65
`sec-edgar`	M	123	0.27	0.46	0.49	0.48
`confluence-md-exporter`	M	206	0.51	0.58	0.58	0.78
`toolz`	M	121	0.56	0.61	0.67	0.97
`all-repos`	M	221	0.80	0.82	0.83	0.90
`mcp-seq-thinking`	M	36	0.72	0.89	0.89	1.00
`bedrock-agentcore-sdk`	M	356	0.68	0.92	0.92	0.95
`creosote`	M	63	0.92	0.92	0.92	0.98
`redbeat`	H	63	0.59	0.59	0.60	0.71
`pyrlp`	H	120	0.53	0.64	0.65	0.71
`apiflask`	H	142	0.47	0.68	0.69	0.81
`click`	H	704	0.46	0.68	0.69	0.79
`marshmallow-sqlalchemy`	H	38	0.68	0.76	0.76	0.90
`scrapyrt`	H	25	0.72	0.80	0.80	0.95
`xacro`	H	140	0.84	0.84	0.85	1.00
`autoray`	H	50	0.86	0.74	0.88	0.97
`python-neutronclient`	H	47	0.92	0.94	0.94	0.96
Median over repositories			0.62	0.75	0.78	

How much is missing. The union of both static graphs contains a median of 0.78 of the observed calls per repository, from 0.45 to 0.94. Pyright alone reaches 0.75 and PyCG 0.62; Pyright finds more of the observed calls in 16 of 20 repositories, so in this corpus the type checker is the stronger call-graph source. Pooled over 16,514 call-resolution items (4,448 of them observed calls), the union has precision 0.91 and recall 0.73 (PyCG 0.93/0.57, Pyright 0.91/0.71). Plain name matching, which treats every name on an executed line as a call, has precision 0.27: most names in code that ran are not calls that happened.

Where it misses. Of the 1,218 observed calls the union misses, 80% go to a method whose name is shared by several functions in the package, that is, dispatch over a class hierarchy or over unrelated classes with a common method name, and 20% are names written without call syntax, such as property reads and functions passed as values. The worst repository illustrates the first pattern: `ShivyC`, a C compiler, defines `make_il` on 29 syntax-tree classes and `make_asm` on 20, and the static union holds fewer than half of its observed calls. A further 1,033 observed calls (19% of all calls) reach a function never named in the calling function, typically a callback stored elsewhere; no resolver that reads the caller's code can recover these.

Dynamism did not predict recall. The corpus was stratified by an AST score of reflective and dynamic constructs, yet median union recall is similar across strata: 0.79 (n=3) for LOW, 0.75 (n=8) for MED and 0.76 (n=9) for HIGH. The constructs such scores count, reflection and dynamic imports,

TABLE III
CALL RESOLUTION ON THE LLM SAMPLE. TOP: EACH RESOLVER ON THE ITEMS IT ANSWERED. BOTTOM: THE ITEMS EVERY LLM ANSWERED, SCORED OUT-OF-FOLD.

Resolver (items)	Precision	Recall	
gpt-oss-20b (3,498)	0.69	0.82	
gpt-oss-120b (3,502)	0.66	0.85	
qwen3.8-27b (3,489)	0.75	0.77	
	Precision	Recall	F1
PyCG ∪ Pyright	0.88	0.67	0.76
majority of the three LLMs	0.72	0.83	0.77
static ∪ any LLM	0.58	0.89	0.70
combiner, folds by caller	0.84	0.73	0.78
combiner, leave one repository out	0.81	0.71	0.76

are not where static analysers lose most calls on real code; ordinary polymorphic dispatch is.

D. RQ4: LLMs as call resolvers

Table III compares the LLMs with the static union on the sampled callers, which are enriched for hard items. Every LLM finds more of the observed calls than static analysis does: recall rises from 0.67 for the static union on the same items to 0.82 for gpt-oss-20b, 0.85 for gpt-oss-120b and 0.77 for qwen3.8-27b. Precision falls from 0.88 to between 0.66 and 0.75: the models claim calls that never happen. The larger gpt-oss model has the highest recall and the lowest precision. On real code, then, the micro-benchmark finding that PyCG outperforms LLMs [4] holds for precision but reverses for recall.

Simply adding every LLM claim to the static graph maximises recall at a large cost in precision. A majority vote of the three LLMs does slightly better than the static union on F1. The learned combiner, trained on 3,485 items from 20 repositories, gives every source a positive weight and discounts names written without call syntax. Scored with folds grouped by caller, it beats every single source. Scored on a repository it has never seen, its F1 falls back to that of the static union: it trades precision for recall rather than improving both. What the combiner learns about one project does not transfer to the next.

V. DISCUSSION

Static call graphs are less settled than their benchmarks suggest. On micro-benchmarks PyCG is precise; on real repositories it fails to terminate on a fifth of the corpus and is seed-dependent on nearly half of the rest. Tools that consume call graphs should treat them as one noisy view, and taking the union of two analysers is cheap.

LLMs are recall tools, not replacements. On hard items the LLMs recover calls that both static analysers miss, mostly through shared method names, but they also assert calls that do not happen. A learned combination helps within the projects it was trained on and not on a new one. For a tool that must work on unseen code, the static union remains the safer default; LLM answers are worth adding where missing a call costs more than a false one, as in impact analysis before an edit.

Call graphs are not reference graphs. In a small pilot on repository-wide renames in `click`, every rename that failed even with runtime-derived callers in context (8 of 8, with an editor that applies exactly what the context shows) failed on a line that names the function without calling it: an entry in a dispatch dictionary, a callback registration, a monkey-patch. A call graph, static or dynamic, does not hold these lines. Code assistants that retrieve context through call graphs need reference edges as well.

VI. THREATS TO VALIDITY

Ground truth is workload-conditional. The tracer sees only what the test suites execute. Recall is measured against the calls the tests made, a subset of the calls the program can make, and precision only on lines that executed; a static edge that no test exercised is neither confirmed nor counted against the tool.

Environments. Static extraction (RQ1, RQ2) used the frozen environments of the triage run. The runtime traces for RQ3 and RQ4 were collected on a second x86-64 host where each repository was installed by the same recipe rather than from the frozen dependency list, so dependency versions may differ; the static graphs are computed from source and are unaffected. `click` and `toolz`, measured earlier, were extracted and traced on an arm64 development machine; both are pure-Python packages, so no compiled dependency enters their graphs.

Tool versions and configuration. We measure one version of each analyser. Pyright ran without dependencies installed. PyCG's 30-minute budget is a choice; a larger budget would finish more repositories, at a cost no interactive tool can pay.

LLM sample and serving. The LLMs answered a sample of callers, enriched for hard items, so their scores are not directly comparable to the static scores on all items; we report every resolver on the same sample. Models served through an API may change; we record the serving model per answer.

Corpus. The corpus excludes repositories that cannot be installed from their root with `pip` or that collect no tests, which biases it against applications and monorepos. The runtime subset favours small, pure-Python packages whose test suites need no external services. The LOW stratum of the runtime subset has only three repositories.

VII. CONCLUSION

On real Python repositories, the standard static call-graph generator fails to terminate on a fifth of the corpus and is non-deterministic on nearly half of the rest, and two static analysers disagree on a third of the edges. Measured against the calls tests actually make, the union of two static analysers finds a median of 0.78 of the calls, and its misses concentrate on shared method names rather than on the reflective constructs usually blamed. LLMs recover many of those calls at a cost in precision, and a learned combination of the two does not generalise to unseen projects. We release the corpus pins, the

extraction and tracing pipelines, and every measurement, so that the numbers can be re-measured as tools change.

Acknowledgment

LLM queries were made through the co-authors' Groq API accounts, with Groq's permission.

Data Availability

Pinned repository list, image digests, dependency-lock hashes, extraction and tracing code, and all results will be released upon publication.

References

[1] S. Ouyang *et al.*, "RepoGraph: Enhancing AI software engineering with repository-level code graph," arXiv:2410.14684, 2024.

[2] V. Salis, T. Sotiropoulos, P. Louridas, D. Spinellis, and D. Mitropoulos, "PyCG: Practical call graph generation in Python," in *Proc. 43rd IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2021, pp. 1646–1657.

[3] I. Bouzenia, B. P. Krishan, and M. Pradel, "DyPyBench: A benchmark of executable Python software," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 338–358, 2024.

[4] A. P. S. Venkatesh *et al.*, "An empirical study of large language models for type and call graph analysis in Python and JavaScript," *Empirical Software Engineering*, 2025.

[5] K. Huang *et al.*, "Scalable and precise application-centered call graph construction for Python," arXiv:2305.05949, 2023.

[6] L. Sui, J. Dietrich, A. Tahir, and G. Fourtounis, "On the recall of static call graph construction in practice," in *Proc. 42nd ACM/IEEE Int. Conf. Software Engineering (ICSE)*, 2020, pp. 1049–1060.

[7] D. Helm, S. Keidel, A. Kampkötter, J. Düsing, T. Roth, B. Hermann, and M. Mezini, "Total recall? How good are static call graphs really?" in *Proc. 33rd ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2024.

[8] Microsoft, "Pyright: Static type checker for Python," https://github.com/microsoft/pyright, version 1.1.411.

[9] M. Shannon, "PEP 669 – low impact monitoring for CPython," https://peps.python.org/pep-0669/, 2022.

[10] codellm-devkit, "codeanalyzer-python: pinning PYTHONHASHSEED for PyCG," https://github.com/codellm-devkit/codeanalyzer-python, commit ae2df74.